

CS112: Friday 2 October, 12:15 to 1:20

Wednesday our `append` stopped copying everything. It doubles the room when it runs out, and the debugger showed the address in the watch window changing under us as the array moved.

Today: nothing ever gives that array back. And copying a `PyList` copies the address, so two objects end up owning one block. Three functions fix both.

TIME	BEAT
12:15	Daily quiz
12:20	1. It leaks right now
12:34	2. The copy that bites
12:50	3. Our own block: the copy constructor
13:04	4. <code>operator=</code> and the rule of three
13:16	5. Close

Beat 1. It leaks right now (14 min)

Talk to your neighbor. A `PyList` goes out of scope at the end of a function. What happens to the array it asked for?

The answer comes from running it.

```
g++ -std=c++17 -g -fsanitize=address -o leak leak.cpp PyList.cpp
&& ./leak
```

SUMMARY: AddressSanitizer: 20 byte(s) leaked in 1 allocation(s).

What one line fixes this, and where does it go?

```
PyList::~PyList() { delete [] myArray; myArray = nullptr; }
```

The compiler writes a destructor for you and it destroys the **members**. It destroys the pointer. It does not touch what the pointer points at.

If your class calls `new`, your class defines a destructor.

The sanitizer runs again. Clean.

Beat 2. The copy that bites (16 min)

```
PyList a;  
a.append(3);  
a.append(7);
```

```
PyList b = a;           // what exactly got copied?
```

Talk to your neighbor. `b` is a new object with its own three members. What is in them?

On the board: two objects on the stack, one arrow out of each, both arrows landing on the same block.

```
free(): double free detected in tcache 2  
Aborted (core dumped)
```

Two faults from one line. `b[0] = 9` changes `a[0]`. The second destructor frees a block that is already gone, which is the crash above. Same crash as the lab, returning a list by value.

Beat 3. Our own block: the copy constructor (14 min)

What has to happen instead, for `b` to be independent?

```
PyList::PyList(const PyList &orig) {  
    mySize      = orig.mySize;  
    myCapacity  = orig.myCapacity;  
    myArray     = new Item[myCapacity]; // our own block  
    for (int i = 0; i < mySize; ++i)  
        myArray[i] = orig.myArray[i]; // copy the values  
}
```

Same moves as Wednesday's growth code: allocate, copy across, own it. Now each object has its own block.

It runs in three places, and only the first one looks like a copy:

```
PyList b = a;           // 1. written down  
void show(PyList v);     // 2. passed by value  
PyList makeList();       // 3. returned by value
```

Which of the three have we already written without noticing?

Beat 4. `operator=` and the rule of three (12 min)

`b = a;` on an object that already exists is not the copy constructor at all. It is assignment, and the default one copies the address the same way.

```
PyList& PyList::operator=(const PyList &rhs) {  
    if (this != &rhs) { // a = a would free its own array  
        delete [] myArray;  
        mySize      = rhs.mySize;  
        myCapacity  = rhs.myCapacity;  
        myArray     = new Item[myCapacity];  
        for (int i = 0; i < mySize; ++i)  
            myArray[i] = rhs.myArray[i];  
    }  
    return *this;       // so that a = b = c works  
}
```

The rule of three. A class that needs one of the destructor, the copy constructor or the assignment operator needs all three. All three answer the same question: who frees this block?

Beat 5. Close (4 min)

`PyList` can now be copied, assigned and destroyed without losing an array or freeing one twice.

Monday: `PyList` stops caring whether it holds ints.